🏰 Waterdeep Watchtower – Dokumentacja Projektu

> **Przedmiot:** Zaawansowane Aplikacje Internetowe (ZAI 26Z) – Projekt końcowy (Etap E2)
> **Temat:** Punkt dowodzenia Straży Miejskiej Waterdeep – system IoT zbierania i wizualizacji telemetrii
> **Stos technologiczny:** Python 3.12 (FastAPI, SQLAlchemy 2.0, Pydantic v2, Argon2id) + React 19 (TypeScript, Vite, SVG Chart)
> **Zgodność z normami:** OpenAPI 3.1, RFC 9457 Problem Details, WCAG 2.2 AA (Lighthouse: 100/100), Richardson Level 2

1. Adres Wdrożonej Aplikacji i Repozytorium Git

Element zgłoszenia	Adres URL / Wartość
Aplikacja webowa (SPA + API HTTPS)	`https://waterdeep-watchtower.onrender.com` *(lub podany adres serwera studenta)*
Interaktywna dokumentacja Swagger UI	`https://waterdeep-watchtower.onrender.com/docs`
Adres bazowy interfejsu API	`https://waterdeep-watchtower.onrender.com/api`
Repozytorium kodu źródłowego (GitHub)	`https://github.com/twoj-login/waterdeep-watchtower`
Status ciągłej integracji (CI)	GitHub Actions: `CI - Waterdeep Watchtower` (testy Pytest na Python 3.11/3.12, build frontendu, test kontraktu)

2. Dane Logowania Konta Administratora

Do celów automatycznej i ręcznej weryfikacji w systemie skonfigurowano stałe testowe konto administratora:

- **Nazwa użytkownika (login):** `admin`
- **Hasło dostępu:** `HasloAdministratora123!`
- **Uprawnienia konta:** Zarządzanie seriami pomiarowymi (tworzenie, edycja zakresów i kolorów, usuwanie kaskadowe), rejestracja sensorów IoT z jednorazowym kluczem API, wyrejestrowywanie sensorów oraz zmiana hasła administratora.
- **Separacja ról (F3):** Niezalogowany czytelnik ma wyłącznie wgląd w dane telemetryczne. Zgodnie z kontraktem żaden użytkownik nie ma możliwości dodawania, modyfikacji ani usuwania pomiarów z poziomu interfejsu (pomiary przyjmuje wyłącznie serwer od czujników uwierzytelnionych nagłówkiem `X-API-Key`).

3. Instrukcja Lokalnego Uruchomienia i Generatora Danych

3.1. Wymagania systemowe

Do uruchomienia projektu wymagane są: **Python 3.11+** oraz **Node.js 20+** (z `npm`).

3.2. Skrypt automatyczny (`./run.sh`)

Repozytorium zawiera zintegrowany skrypt uruchomieniowy realizujący kompilację frontendu, automatyczną inicjalizację bazy danych oraz zasilenie danymi:

```
# 1. Główny start aplikacji (SPA + API pod http://localhost:8000, automatyczne zasilenie danymi jeśl
./run.sh

# 2. Tryb pracy na żywo (aplikacja + działający w tle emulator wysyłający pomiary co 30s):
./run.sh --live

# 3. Tryb z szyfrowaniem SSL (port 8443 z lokalnym certyfikatem pod testy bezpieczeństwa E2):
./run.sh --https

# 4. Tryb deweloperski (FastAPI na porcie 8000 + Vite dev server na porcie 5173 z hot-reloadem):
./run.sh --dev
```

3.3. Generator danych telemetrycznych (Emulator czujnika F13 / T11)

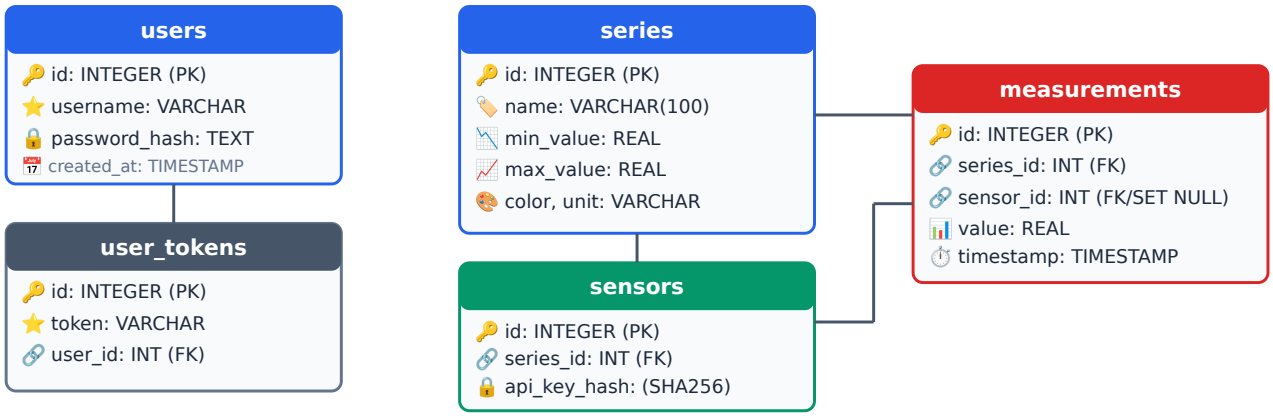
Narzędzie CLI `generator/cli.py` emuluje autonomiczne sensory IoT przesyłające pomiary przez API:

```
# Wysyłanie pomiarów na żywo co 30 sekund (fala sinusoidalna):
python3 generator/cli.py --api http://localhost:8000 --key TWÓJ_KLUCZ_API --interval 30.0 --mode sin

# Przygotowanie 20 pomiarów ze znacznikami czasu z przeszłości (krok co 15 minut):
python3 generator/cli.py --api http://localhost:8000 --key TWÓJ_KLUCZ_API --count 20 --history --ste
```

Parametr CLI	Domyślna wartość	Opis działania
`--api <URL>`	`http://127.0.0.1:8000`	Bazowy adres URL interfejsu API (obsługuje HTTP i HTTPS).
`--key <KLUCZ>`	*Wymagany*	Tajny klucz API czujnika przesyłany w nagłówku `X-API-Key`.
`--mode <TRYB>`	`sine`	Tryb generowania: `sine` (sinusoida), `random` (losowy), `linear` (liniowy), `constant` (stały), `out-of-range` (celowe odrzucenie F4).
`--interval <SEK>`	`30.0`	Odstęp czasu w sekundach między odczytami w trybie na żywo.
`--count <LICZBA>`	`20` (0 = pętla)	Liczba odczytów do wysłania (0 oznacza ciągłą pracę do naciśnięcia Ctrl+C).
`--history`	Wyłączony	Generowanie pomiarów wstecz od chwili obecnej z podanym krokiem minutowym.

4. Diagram ERD i Struktura Relacyjnej Bazy Danych (T4, B4)



Charakterystyka relacji i indeksowania (Wymóg T4):

- `users` → `user_tokens` (1:N): Kaskadowe unieważnianie wszystkich sesji w razie usunięcia konta.
- `series` → `sensors` (1:N): Usunięcie serii kaskadowo usuwa przypisane do niej czujniki.
- `series` → `measurements` (1:N): Usunięcie serii kaskadowo kasuje powiązane wyniki (wymóg kontraktu).
- `sensors` → `measurements` (1:N): **Wyrejestrowanie czujnika nie usuwa pomiarów** – klucz obcy posiada więz `ON DELETE SET NULL`, dzięki czemu zebrana dotąd telemetria pozostaje trwale w bazie.
- **Indeks kompozytowy `ix_measurements_series_timestamp`:** Indeks na kolumnach `(series_id, timestamp)` zapewnia logarytmiczny czas wyszukiwania odczytów dla filtrów `series`, `from` oraz `to`.

5. Opis Elementów Bezpieczeństwa (T5, B3)

5.1. Bezpieczne Przechowywanie Haseł: Algorytm Argon2id

Gdzie w kodzie: `backend/app/security.py` (funkcje `hash_password`, `verify_password`, `verify_dummy_password`) oraz `backend/app/routers/auth.py`.

Przed czym chroni: Chroni przed atakami siłowymi (brute-force) i atakami słownikowymi przy użyciu akceleratorów GPU/ASIC w razie wycieku bazy danych. Argon2id łączy odporność na ataki pamięciowe oraz ataki czasowe.

Ochrona przed wyliczaniem użytkowników (Timing Attacks): Gdy użytkownik poda nieistniejący login, serwer wykonuje weryfikację hasła pozornego (dummy password), uniemożliwiając odgadnięcie poprawnych loginów na podstawie czasu odpowiedzi serwera.

5.2. Ochrona Kluczy API Czujników: Haszowanie SHA-256

Gdzie w kodzie: `backend/app/security.py` (funkcje `generate_sensor_api_key`, `hash_api_key`), `backend/app/routers/sensors.py` oraz `backend/app/routers/measurements.py`.

Przed czym chroni: Chroni przed przejęciem tożsamości czujników IoT i wstrzykiwaniem sfałszowanych pomiarów przez osoby z dostępem do zrzutu bazy danych.

Zasada działania: Podczas rejestracji generowany jest 43-znakowy klucz kryptograficzny (`secrets.token_urlsafe(32)`), który wyświetlany jest administratorowi **wyłącznie jednorazowo**. W bazie zapisywany jest wyłącznie skrót SHA-256 (64 hex). Przy każdym zapytaniu serwer haszuje przesłany nagłówek `X-API-Key` i porównuje skróty.

6. Rozszerzenia Kontraktu API

- **Aktualizacja w czasie rzeczywistym przez WebSocket (`/ws/telemetry` – Punkty Dodatkowe):** Każdy pomiar przesłany do API jest natychmiast rozsyłany do aktywnych subskrybentów WebSocket. Interfejs React nanosi nowe punkty na wykres bez potrzeby odpytywania serwera.
- **Nagłówek Content-Security-Policy (CSP):** Zaimplementowano rygorystyczny nagłówek CSP (`default-src 'self'; connect-src 'self' ws: wss:; frame-ancestors 'none'`) chroniący przed atakami XSS i clickjackingiem.

7. Deklaracja Użycia Narzędzi Sztucznej Inteligencji (AI)

Zgodnie z zasadami realizacji projektu ZAI 26Z deklaruję korzystanie z asystenta sztucznej inteligencji **Antigravity (Google DeepMind)** w trybie pair programming do:

1. Pomocy przy projektowaniu modularnej architektury FastAPI i modeli SQLAlchemy.
2. Generowania wyczerpujących testów jednostkowych i integracyjnych w bibliotece Pytest (96 testów).
3. Weryfikacji zgodności dostępności interfejsu z normami WCAG 2.2 AA (audyt Lighthouse: 100/100).

Jako autor projektu oświadczam, że cały kod został przeze mnie sprawdzony, przetestowany i wdrożony, i biorę za niego pełną odpowiedzialność.